Linux & the Command Line

Unit 04 — Technical Foundations

Almost every server you'll ever test, and almost every tool you'll ever use, runs on Linux. Time to learn to drive it.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Learning objectives

By the end of this unit you can:

- **Explain** why Linux and the CLI matter in security.
- **Navigate** the filesystem with `pwd`, `ls`, `cd`.
- **Manage files**: `touch`, `cat`, `less`, `cp`, `mv`, `rm`, `mkdir`.
- **Find** files and text: `find`, `locate`, `which`, `grep`.
- **Read & change** permissions (`rwx`, `chmod`, `chown`); explain `sudo` / `root`.
- **Inspect** users & processes: `whoami`, `id`, `ps`, `top`, `kill`.
- **Combine** commands with pipes and redirection (`|`, `>`, `>>`, `<`).
- **Complete** OverTheWire Bandit levels 0–~12 and log each command.

Why Linux? Why the command line?

- Most **servers** on the internet run Linux — so most **targets** do too.
- The pentester's toolkit (**Kali Linux**) is built on it.
- The CLI is **faster, scriptable, and repeatable** — perfect for automation.
- No mouse needed: you can work over a remote connection (**SSH**).

> You can't drive a security tool you can't drive the operating system under it.

Shell, terminal, command line

Term	What it is
Terminal	The window/app you type into
Shell	The program (e.g., **Bash**) that reads and runs your commands
Command line (CLI)	Controlling the computer by typing, not clicking

You type a command → the shell runs it → you read the output → try again.

The filesystem hierarchy

Everything lives in one tree that starts at `/` (the **root**).

```
/
├── home/       your files live here
│   └── student/
├── etc/        system configuration
├── bin/        programs
└── tmp/        temporary files
```

- **Absolute path:** the full address from `/` → `/home/student/notes.txt`
- **Relative path:** from where you are right now → `notes.txt`

Getting around

```
pwd             # print working directory — "where am I?"
ls              # list what's here
ls -la          # long format, including hidden files
cd /etc         # change directory (absolute)
cd ..           # go up one level
cd ~            # go to your home directory
```

> The **working directory** is the folder you're "in" right now.

Working with files

```
touch notes.txt          # create an empty file
mkdir practice           # make a directory
cat notes.txt            # print a file to the screen
less bigfile.txt         # view one screen at a time (q to quit)
head / tail file.txt     # first / last lines
cp a.txt b.txt           # copy
mv b.txt c.txt           # move or rename
rm c.txt                 # remove (DELETE)
```

`rm` **has no recycle bin.** Deleted is gone. Read before you delete.

Finding things

```
find / -name "flag.txt"   # live search by name
locate passwd             # fast search of an index
which python3             # where is this command's program?
grep "password" file.txt  # search INSIDE files for text
grep -r "flag" .          # recursive — search a whole folder
```

- `find` / `locate` find **files**.
- `grep` finds **text inside** files (or any input).

Permissions: the rwx model

`ls -l` shows them. Read left to right:

```
-rwxr-xr--   owner: rwx   group: r-x   other: r--
 |└┘└┘└┘
 | |  |  └── other (everyone else)
 | |  └───── group
 | └──────── owner
 └─────────── file type
```

- **r** = read · **w** = write · **x** = execute
- Three groups: **owner**, **group**, **other**

Changing permissions & power

```
chmod +x script.sh     # make a file executable
chmod 644 notes.txt    # set rw-r--r--
chown student file.txt # change the owner
```

- **root** = the all-powerful administrator account.
- **sudo** = "superuser do" — run **one** command as administrator.

> Use `sudo` only when you truly need it. That's **least privilege**.

Users & processes

```
whoami     # which user am I?
id         # my user and group IDs
ps         # list my processes
top        # live view of processes & resource use (q to quit)
kill 1234  # stop the process with PID 1234
```

A **process** is just a running program. Each has a **PID** (process ID).

Pipes & redirection

```
ls -l | grep txt          # send ls output INTO grep
cat data.txt | head       # chain commands together

echo "hi" > out.txt       # > overwrite a file
echo "more" >> out.txt    # >> append to a file
sort < names.txt          # < take input from a file
```

- **Pipe** `|` = output of one command becomes input of the next.
- This is how small tools combine into powerful workflows.

Getting help (the most important skill)

```
man find        # full manual (press q to quit)
grep --help     # quick options summary
apt --help
```

- `man` and `--help` let you figure out *any* command on your own.
- Package basics: `sudo apt update`, `sudo apt install <tool>` — a **package manager** installs and updates software.

> When stuck: read the manual *before* asking. That's the pro habit.

Ethics & Authorization

Bandit is authorized. Your school network is not.

OverTheWire publishes **Bandit** *specifically* so you can practice — that's your written permission, and your scope is the Bandit servers only.

The **same** `ssh`, `find`, and `grep` commands would be **illegal** pointed at the school's file server, a classmate's account, or any machine outside this lab.

Key vocabulary

Term	Meaning
Shell / Terminal	The program that runs commands / the window you type in
Path	A file's address (**absolute** from `/`, or **relative**)
Permissions (rwx)	Read / write / execute for owner, group, other
root / sudo	The admin account / run one command as admin
Process / PID	A running program / its ID number
Pipe `|`	Feeds one command's output into the next
SSH	Encrypted remote login to another machine

Lab launch: OverTheWire Bandit

Platform: OverTheWire **Bandit** — a free, pre-authorized SSH wargame.

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
# password: bandit0   (note: port 2220, NOT 22)
```

Goal of each level: find the password for the *next* level using the right command. Then log in as `bandit1`, `bandit2`, …

You'll use: `cat`, `ls -a`, `file`, `find`, `grep`, `sort | uniq -u`, `strings`, `base64 -d`, `tr` (ROT13).

> **Log the command and your reasoning — not just the password.** The skill is the point.

Recap

- Linux runs the servers and the tools — the CLI is the foundation.
- **Navigate:** `pwd`, `ls`, `cd` · **Files:** `cat`, `cp`, `mv`, `rm`, `mkdir`
- **Find:** `find`, `grep` · **Permissions:** `rwx`, `chmod`, `sudo`
- **Inspect:** `whoami`, `ps`, `top`, `kill`
- **Combine:** pipes `|` and redirection `>`, `>>`, `<`
- **Get help:** `man` and `--help` — figure anything out
- Same skills, different target = the line between **practice** and **crime**.

Exit ticket & discussion

1. What command tells you **where you are**, and what lists **what's there**?
2. What's the difference between `cat` and `less`? Why is `rm` dangerous?
3. What does the `|` symbol do? `>` vs `>>`?

Discuss: On Bandit you hunt for a password file and that's allowed. Run the *exact same commands* on the school server — what changes, and why is one a crime?

Submit your Bandit progress log: one entry per level (goal, command, what you learned).